Microservices

Apertura: martes, 7 de abril de 2026, 00:19
Cierre: lunes, 20 de abril de 2026, 23:59

Assignment - EXPERIMENTAL

Building a Secure Twitter-like Application with Microservices and Auth0

Objective: Design and implement a simplified Twitter-like application that allows authenticated users to create short posts (maximum 140 characters) in a single public stream/feed. The project must start as a Spring Boot monolith, evolve into serverless microservices on AWS, and be fully secured using **Auth0**.

Instructions:

Work in **groups of maximum 3 students**.

You must complete the following tasks:

1. **API Design and Monolith Development** Design a **RESTful API** and implement a **Spring Boot monolithic application** that allows authenticated users to create posts of up to **140 characters**. All posts are stored and displayed in a **single public stream** (similar to a Twitter feed). Consider at least the following main entities:
    - **User**
    - **Post**
    - **Stream** (a single global stream of posts)
2. **API Documentation** Include a complete **Swagger (OpenAPI)** definition of your API in the Spring Boot monolith. The Swagger UI should be accessible (e.g., at /swagger-ui.html or /swagger-ui/index.html) and clearly document all endpoints, request/response models, parameters, and security requirements (JWT Bearer token).
3. **Frontend Application** Develop a **JavaScript-based web application** (you may use vanilla JavaScript, React, Vue.js, or Angular) that consumes the backend API. The frontend should allow users to:
    - Log in / Log out
    - Create new posts
    - View the public stream of posts Deploy the frontend as a **static website on Amazon S3** and ensure it is **publicly accessible** over the internet.
4. **API Security (Mandatory)** Secure the entire RESTful API using **Auth0** as the identity and authorization provider.
    - Create a **Single Page Application (SPA)** client in Auth0 for the frontend.
    - Define a dedicated **API** in Auth0 with a unique **Audience** for the backend.
    - Implement authentication and authorization using **JWT access tokens** issued by Auth0.
    - The **frontend** must integrate with Auth0 using the official **Auth0 React SDK** (strongly recommended) or **Auth0 SPA JS SDK**. It should handle login (redirect or popup), logout, silent token refresh, and secure token management.
    - The **Spring Boot backend** must be configured as an **OAuth2 Resource Server**. It should validate JWT tokens using Auth0's issuer URI (https://YOUR-DOMAIN.auth0.com/) and enforce the correct **Audience**.
    - Protect your endpoints as follows:
        - **Public endpoints** (no authentication required): e.g. GET /api/posts or GET /api/stream — to read the public post stream.

        - **Protected endpoints** (require valid JWT): e.g. POST /api/posts — only authenticated users can create posts.
        - Add a protected endpoint to retrieve the current user's information: **GET /api/me**.
    - (Recommended) Define and use **scopes** such as read:posts, write:posts, and read:profile for granular authorization.
5. **Testing** Thoroughly test the complete web application (secure frontend + protected backend).
6. **Migration to Microservices** Refactor the monolith into **at least three independent microservices**. Suggested services:
    - User Service / Authentication
    - Posts Service
    - Stream / Feed Service

    Implement each microservice using **AWS Lambda** (serverless).
7. **Deployment on AWS** Deploy all microservices on **AWS Lambda** (with API Gateway if needed). Ensure the frontend hosted on S3 can securely consume the microservices through Auth0-protected endpoints.
8. **Deliverables** Submit everything in a **GitHub repository** (one repository per group is recommended):
    - All source code (monolith, microservices, and frontend).
    - A comprehensive **README.md** that includes:
        - Project description and final architecture overview.
        - Architecture diagram (showing evolution from monolith to microservices + Auth0 security flow).
        - Clear setup and local execution instructions.
        - Report of tests performed.
        - Link to the live frontend deployed on Amazon S3.
        - Link to the **Swagger UI** of the monolith (if still deployed) or a screenshot/export of the OpenAPI specification.
        - A **video demonstration** (recommended 5–8 minutes) showing the fully working application: login with Auth0, creating posts, viewing the stream, retrieving user info via /api/me, and a brief explanation of the architecture and security configuration.

Important Notes:

- Using **Auth0** for API security is **mandatory**.
- The **Swagger / OpenAPI** documentation is **mandatory** for the monolith phase.
- Do not commit sensitive credentials, Auth0 secrets, or API keys to GitHub. Use environment variables or AWS Secrets Manager where appropriate.
- Focus on clean code, proper error handling, security best practices, and good user experience.

Good luck with the assignment!

Editar entrega Borrar entrega

Estado de la entrega

Estado de la entrega	Enviado para calificar
Estado de la calificación	Sin calificar
Tiempo restante	La tarea fue enviada 2 días 17 horas antes de la fecha límite
Última modificación	sábado, 18 de abril de 2026, 06:10

Texto en línea	+ https://github.com/JAPV-X2612/AREP-laboratory-7-microservices
Comentarios de la entrega	Comentarios (0)

Escuela Colombiana de Ingeniería Julio Garavito